# Data

Data is a collection of raw facts, figures, numbers, text, images, or observations that can be processed to get meaningful information.

# Types of Data

**Structured Data**– Structured data is organized data stored in rows and columns, making it easy to search, manage, and analyze.

**Example**- Student records

**Unstructured Data**– Unstructured data is data that is not organized in rows and columns and does not follow a fixed format.

**Example**-Photos, videos, audio files

**Semi-Structured Data** -Semi-structured data is partially organized data that does not follow a strict table format but contains tags or labels for organization.

**Example**-JSON files, XML files, and emails.

# Data Science

Data Science is a process of using data to understand problems, predict future outcomes, and make better decisions with the help of computers, mathematics, and AI techniques.

# Features of Data Science

1. Collects and analyzes data
2. Helps in decision-making
3. Uses statistics and programming
4. Finds patterns and insights
5. Handles large amounts of data

# Scope of Data Science with AI & ML

1. Healthcare
2. Banking & Finance
3. E-Commerce

4. Social Media
5. Cyber Security
6. Education

## Machine Learning

Machine Learning means teaching computers to learn from data and improve automatically without writing every instruction manually

# Features of Machine Learning

1. Learns from data automatically
2. Improves performance over time
3. Makes predictions and decisions
4. Reduces manual work
5. Handles large amounts of data
6. Finds hidden patterns in data

# Types of Machine Learning

1. **Supervised Learning**- Supervised Learning is a type of Machine Learning in which the model learns using labeled data. **Example**- Email spam detection
2. **Unsupervised Learning**- Unsupervised Learning works with unlabeled data. The machine tries to find hidden patterns, groups, or relationships in the data by itself. **Example**- Product recommendations Data clustering
3. **Reinforcement Learning**- Reinforcement Learning is a learning method where the machine learns through rewards and penalties. **Example**- Self-driving cars Robot movement

## Deep Learning

Deep Learning is a type of Machine Learning that uses artificial neural networks to learn from large amounts of data and solve complex problems automatically.

Deep Learning teaches computers to think and learn in a way similar to the human brain using multiple layers of learning.

**Example**-

1. Face unlock in smartphones
2. Voice assistants like Siri and Alexa

3. Self-driving cars
4. YouTube recommendations

# Types of Deep Learning

1. ANN – Basic neural network for learning data patterns.
2. CNN – Used for image and video processing.
3. RNN – Used for text and speech data.
4. LSTM – Remembers long-term sequence information.
5. GAN – Generates new images and content.
6. Autoencoder – Used for data compression.
7. DBN – Used for pattern recognition.

# Features of Deep Learning

1. Automatic learning
2. High accuracy
3. Uses neural networks
4. Handles large data
5. Works with images, audio, and text
6. Supports AI automation
7. Improves with more data

# Python

**Python** is a high-level, interpreted programming language that is easy to learn and used for web development, data science, artificial intelligence, automation, and software development.

## Properties of Python

1. Simple and easy to learn
2. High-level programming language
3. Interpreted language
4. Object-oriented language
5. Platform independent
6. Open-source and free
7. Dynamically typed
8. Supports multiple programming paradigms

9. Large standard library
10. Portable and flexible

## Variables

A variable is container used to store data or variable in memory.

```
name="riya"
print(name)
```

```
riya
```

## Variable name rules:-

1. Variable name must start with a letter or underscore (_)
2. It cannot start with a number
3. Special symbols are not allowed except underscore (_)
4. Variable names are case-sensitive (name and Name are different)
5. Keywords cannot be used as variable names
6. Spaces are not allowed in variable names
7. Use meaningful variable names for better readability

## Data types in Python

8. Integer (int)
9. Float (float)
10. Complex (complex)
11. String (str)
12. List (list)
13. Tuple (tuple)
14. Set (set)
15. Dictionary (dict)
16. Boolean (bool)

**note**:-types of data are zero

# string

String is sequence of character used to store text. It can contain:-Letters,Numbers, Symbols,Spaces.

## Functions in Python

1. Type():- It show data types of string

```
name="upflairs"
print("this is my first string :-",name)
print("this is my first string :-",type(name)) ## type function show data type

this is my first string :- upflairs
this is my first string :- <class 'str'>
```

1. len():- It show the length of string

```
name="upflairs"
print("len of my first string",len(name)) ###find the len of my string

len of my first string 8
```

1. upper():- convert into uppercase

```
company_name= "upflairs pvt ltd"
upper_case=company_name.upper()
print("upper case 😊😊",upper_case)

upper case 😊😊 UPFLAIRS PVT LTD
```

1. lower():- convert into lowercase

```
company_name= "UPFLAIRS PVT LTD"
lower_case=company_name.lower()
print("lower case :-",lower_case)

lower case :- upflairs pvt ltd
```

1. casefold():- used to convert a string into lowercase in a more aggressive way than lower(), mainly for case-insensitive text comparison.

```
company_name= "UPFLAIRS PVT LTD"
lower_case=company_name.casefold()###task2
print(lower_case)

upflairs pvt ltd
```

1. title():- used to converts the first letter of each word into a capital letter.

```
company_name="upflairs pvt ldt"
print(company_name.title())
```

```
Upflairs Pvt Ldt
```

1. capatalize():- usedd to converts the first letter of the string into a capital letter and all other letters into lowercase

```
company_name="upflairs pvt ldt"
print(company_name.capitalize())

Upflairs pvt ldt
```

1. count():- used to counts how many times a value appears.

```
name= "riya"
c= name.count('i')
print(c)

1
```

1. index():- it means accessing a single items from list, tuple or string using its position number. counting starts from zero.

   note:- position number start from one

```
name= "riya"
print(name.index('y'))

2
```

# Indexing & Slicing

- **Indexing**:- using index number wee find the letter in the string

```
name="upflairs"
print(name[0])
print(name[5])

u
i
```

- **Slicing**:- It means extracting a part of sequence.

```
name="upflairs"
print("Slicing:-",name[0:4])

Slicing:- upfl

# merge two string
name ="riya"
last_name="joshi"
print("add two string:-",name+ last_name)
print("add two string:-",name,last_name)  #it is used for space
```

```
between  two string
print("add two string:-",name+" "+ last_name)
```

```
add two string:- riyajoshi
add two string:- riya joshi
add two string:- riya joshi
```

```
#for multiple string
name ="riya"
print(name*2)
```

```
riyariya
```

```
# for multiple lines use (""  """)
intro = """Hi, I am Riya Joshi. I am a second-year B.Tech CSE student
with an interest in web development and programming. I know HTML, CSS,
and JavaScript, and I am currently learning Data Structures and
Algorithms. I have also completed DSA training and enjoy improving my
technical skills. I am hardworking, eager to learn new technologies,
and want to build a successful career in the IT field."""

print(intro)
```

```
Hi, I am Riya Joshi. I am a second-year B.Tech CSE student with an
interest in web development and programming. I know HTML, CSS, and
JavaScript, and I am currently learning Data Structures and
Algorithms. I have also completed DSA training and enjoy improving my
technical skills. I am hardworking, eager to learn new technologies,
and want to build a successful career in the IT field.
```

# f-string

In Python, f is used before a string to create an f-string (formatted string).

It allows you to directly put variables inside a string using {}.

```
name="riya"
print(f"my name is {name} and i am from jaipur")
```

```
my name is riya and i am from jaipur
```

# r-string

r is used to create a raw string in Python.

A raw string is a string in Python where backslashes  are treated as normal characters.

```
path= r"file:///C:/Users/RIya/Documents/DAA%20lab.pdf"
print(path)
```

```
file:///C:/Users/RIya/Documents/DAA%20lab.pdf
```

# List Data Type

- A list is used to store multiple values in one variable.
- Lists are hetrogeneous.
- Lists are written using square brackets [].
- A list can contain numbers, strings, or mixed data types.
- Lists are ordered.
- Lists are changeable (mutable).
- Duplicate values are allowed.
- Elements are accessed using index numbers.

## Functions in List

1. append() :- used to add a new element at the end of a list.

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.append("upflairs")
print(lst)
```

```
[1, 2, 2, 3, 4, 5, 'riya', 2.3, '😊😊😊', 'upflairs']
```

1. insert() :- Used to add an element at a specific position in a list

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.insert(0,"upflairs")
print(lst)
```

```
['upflairs', 1, 2, 2, 3, 4, 5, 'riya', 2.3, '😊😊😊']
```

1. remove() :- Used to remove a specific element from a list.

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.remove(1)
print(lst)
```

```
[2, 2, 3, 4, 5, 'riya', 2.3, '😊😊😊']
```

1. pop() :- Used to remove an element using its index position.

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.pop(5)
print(lst)
```

```
[1, 2, 2, 3, 4, 'riya', 2.3, '😊😊😊']
```

1. extend() :- Used to add multiple elements to a list.

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.extend([1,0])
print(lst)
```

```
[1, 2, 2, 3, 4, 5, 'riya', 2.3, '😊😊😊', 1, 0]
```

1. clear() :- Used to remove all elements from a list.

```
lst=[1,2,2,3,4,5, "riya",2.3,"😊😊😊"]
lst.clear()
print(lst)
```

```
[]
```

1. max() :- Used to find the largest value.

```
lst=[1,4,4,4,3,2,6]
print(max(lst))
```

```
6
```

1. min() :- Used to find the smallest value.

```
lst=[1,4,4,4,3,2,6]
print(min(lst))
```

```
1
```

1. sum() :- Used to calculate the total sum of elements.

```
lst=[1,4,4,4,3,2,6]
print(sum(lst))
```

```
24
```

1. len() :- Used to find the number of elements.

```
lst=[1,4,4,4,3,2,6]
print(len(lst))
```

```
7
```

1. update() :- Used to add or update elements in a set or dictionary.

```
lst=[1,4,4,4,3,2,6]
lst[0]="hello"
print(lst)
```

```
['hello', 4, 4, 4, 3, 2, 6]
```

1. sort() :- Used to arrange elements in ascending order.

```
lst=[1,4,4,4,3,2,6]
lst.sort()
print(lst)

[1, 2, 3, 4, 4, 4, 6]
```

1. reverse() :- Used to reverse the order of elements.

```
lst=[1,4,4,4,3,2,6]
lst.reverse()
print(lst)

[6, 2, 3, 4, 4, 4, 1]
```

1. copy() :- creates the copy of the list.

```
lst=[1,4,4,4,3,2,6]
lst.copy()
print(lst)

[1, 4, 4, 4, 3, 2, 6]
```

# Tuple

**Tuple** is a simple way to store multiple items together in one group. Unlike a list, the items in a tuple cannot be changed after it is created.

## key points of python

1. Stores multiple values together
2. Ordered collection (keeps item order)
3. Immutable (cannot be changed after creation)
4. Allows different data types
5. Written using parentheses ()
6. Can contain duplicate values

```
## example
tpl=(1,2,3,4,5,"hello",4.5,"😁😁")
print("this is my first tuple:-",tpl)
print("type of my first tupple:-",type(tpl))
print("length of my first tupple:-",len(tpl))

this is my first tuple:- (1, 2, 3, 4, 5, 'hello', 4.5, '😁😁')
type of my first tupple:- <class 'tuple'>
length of my first tupple:- 8
```

# Indexing and Slicing

**Indexing** it means accessing a single item from a list, tuple, or string using its position number. Counting starts from 0.

**Slicing** it means taking a part of a list, tuple, or string by using a range of indexes.

```
tpl=(1,2,3,4,5,"hello",4.5,"😁😁")
print(tpl[5])
print(tpl[7])
print(tpl[2:5])
```

```
hello
😁😁
(3, 4, 5)
```

# Tuple unpacking

It assigning the elements of a tuple to multiple variables in a single statement.

```
a,b,c=(1,2,3)
print(a)
print(b)
print(c)
```

```
1
2
3
```

# Type Casting

It means converting one data type to another

```
tpl=(1,2,3,4,"hello")
print("my tupple",tpl)
print("type of my tupple",type(tpl))

print("<<<<<<<<<<<tuple convert into list<<<<<<<<<<")

lst=list(tpl)
print("this is my list",lst)
print("type of my list:-",type(lst))

print(".............>add item in list")
lst.append("item add ho chuka ")
print("list ma item add ho chuka hai:-",lst)


print("..................>>>>>>>.list convert into tuple")
```

```
tpl=tuple(lst)
print("my tupple",tpl)
print("type of my tupple",type(tpl))
print ("tuple mai items add ho chuka hai")
```

```
my tupple (1, 2, 3, 4, 'hello')
type of my tupple <class 'tuple'>
<<<<<<<<<<<tuple convert into list<<<<<<<<<<
this is my list [1, 2, 3, 4, 'hello']
type of my list:- <class 'list'>
.............>add item in list
list ma item add ho chuka hai:- [1, 2, 3, 4, 'hello', 'item add ho
chuka ']
..................>>>>>>>.list convert into tuple
my tupple (1, 2, 3, 4, 'hello', 'item add ho chuka ')
type of my tupple <class 'tuple'>
tuple mai items add ho chuka hai
```

# Dictionary

A dictionary is a collection that stores data in key-value pairs. It is used to quickly access data using a key instead of an index.

```
### example
student={ "name":"saloni",
          "class":"3rd year",
          "subject":"python",
          "rollnumber":11,
          "branch":"cse"}
print(student)
print(student.keys())
print(student.values())
print(student.items())
```

```
{'name': 'saloni', 'class': '3rd year', 'subject': 'python',
'rollnumber': 11, 'branch': 'cse'}
dict_keys(['name', 'class', 'subject', 'rollnumber', 'branch'])
dict_values(['saloni', '3rd year', 'python', 11, 'cse'])
dict_items([('name', 'saloni'), ('class', '3rd year'), ('subject',
'python'), ('rollnumber', 11), ('branch', 'cse')])
```

## using get function

```
###example
student={ "name":"saloni",
          "class":"3rd year",
          "subject":"python",
          "rollnumber":11,
```

```
          "branch":"cse"}
print(student.get('name'))
print(student.get('branch'))
print(student.get('rollnumber'))
print(student.get('class'))
```

```
saloni
cse
11
3rd year
```

## Pop Operation

The pop operation is used to remove an element from the notes collection. It is commonly used when deleting the last added note or removing a specific item from a list.

```
#example
student={ "name":"saloni",
          "class":"3rd year",
          "subject":"python",
          "rollnumber":11,
          "branch":"cse"}
print(student.pop('name'))
```

```
saloni
```

## Set Default

The set default method helps initialize a note or value if it does not already exist. This prevents errors and ensures that default information is available when needed.

```
# example
car={"brand":"honda",
         "model":2025,
        "owner":"second owner"}

x= car.setdefault('color','white')
print(x)
```

```
white
```

## Update Operation

The update operation allows modification of existing notes or data. It is useful when editing note content, changing titles, or updating stored information dynamically.

```
car={"brand":"honda",
       "model":2025,
```

```
        "owner":"second owner",
        "car name":["Honda Amaze","Honda Elevate","Honda
City","hondaCity e:HEV","Honda Civic"]}

car['model']=2026
print(car)
```

```
{'brand': 'honda', 'model': 2026, 'owner': 'second owner', 'car name':
['Honda Amaze', 'Honda Elevate', 'Honda City', 'hondaCity e:HEV',
'Honda Civic']}
```

## Adding an Element

Adding an element means inserting a new note into the collection. This is one of the most essential operations in any note-making application, enabling users to store and manage new information easily.

```
car={"brand":"honda",
        "model":2025,
        "owner":"second owner",
        "car name":["Honda Amaze","Honda Elevate","Honda
City","hondaCity e:HEV","Honda Civic"]}


car['car name'].append ("bmw")
print(car)
```

```
{'brand': 'honda', 'model': 2025, 'owner': 'second owner', 'car name':
['Honda Amaze', 'Honda Elevate', 'Honda City', 'hondaCity e:HEV',
'Honda Civic', 'bmw']}
```

# Set

A set is a collection of distinct, well-defined objects (numbers, symbols, shapes, or even other sets).

Elements of a set are written inside curly brackets { } and separated by commas.

```
sat={1,2,3,5}
print("this is my set:-",sat)
print("len of my set:-",len(sat))
print("type of  my set:-",type(sat))
```

```
this is my set:- {1, 2, 3, 5}
len of my set:- 4
type of  my set:- <class 'set'>
```

## Set add()

Adds a single element to the set.

```
sat={1,2,3,5}
sat.add(10)
print(sat)
```

```
{1, 2, 3, 5, 10}
```

## Remove() Method

Removes a specified element from the set.

```
sat={1,2,3,5}
sat.remove(1)
print(sat)
```

```
{2, 3, 5}
```

## Discard() Method

Also removes an element from the set.

```
sat={1,2,3,5}
sat.discard(10)
print(sat)
```

# Set Operations in Python

Set operations are used to compare and combine sets. They are very useful in mathematics, databases, and programming.

1. union()

   The union of two sets contains all unique elements from both sets.

```
s1={1,2,3,4}
s2={3,4,5,6,7}

print(s1|s2)
result=s1.union(s2)
print(result)
```

```
{1, 2, 3, 4, 5, 6, 7}
{1, 2, 3, 4, 5, 6, 7}
```

1. Intersection()

   The intersection of two sets contains only the common elements present in both sets.

```
s1={1,2,3,4}
s2={3,4,5,6,7}
print(s1&s2)
result=s1.intersection(s2)
print(result)

{3, 4}
{3, 4}
```

1. Difference()

   The difference of two sets gives elements that are present in the first set but not in the second set.

```
s1={1,2,3,4}
s2={3,4,5,6,7}
print(s1-s2)

{1, 2}
```

1. Symmetric difference()

   The symmetric difference contains elements that are in either set, but not in both.

```
s1={1,2,3,4}
s2={3,4,5,6}
print(s1^s2)

{1, 2, 5, 6}
```